

HTML SVG

A Practical, From-Zero Learning Guide

Part 10 — SVG with HTML, CSS, JavaScript & React

This part connects SVG to real web development: inline SVG, external files, CSS, JavaScript, accessibility, responsive layouts, sprites and React components.

Goal: Take an SVG from a design or create one yourself and integrate it cleanly into a modern web application.

1. Where SVG Can Live in a Web Page

- Inline inside HTML.
- As an external image with `<img>`.
- As a CSS background.
- Inside `<object>` or `<iframe>` for certain use cases.
- Inside a React component.
- Inside an SVG sprite and referenced with `<use>`.

2. Inline SVG

```
<!DOCTYPE html>
<html>
<body>
  <svg viewBox="0 0 200 100" width="200" height="100">
    <circle cx="100" cy="50" r="30" fill="tomato"/>
  </svg>
</body>
</html>
```

Inline SVG becomes part of the document DOM, making it convenient for styling and interaction.

3. SVG as an Image

```

```

This is simple and often ideal for logos and illustrations. The SVG's internal elements are not directly part of the page DOM.

4. Inline vs img

Inline: excellent CSS access and JavaScript interaction. ****: simple and cacheable, but page CSS/JS cannot directly manipulate the SVG's internal shapes.

5. SVG as a CSS Background

```
.hero {
  width: 300px;
  height: 200px;
  background-image: url("/images/pattern.svg");
  background-size: cover;
}
```

Useful for decorative graphics that do not need semantic content.

6. SVG and CSS Classes

```
<svg class="icon" viewBox="0 0 24 24">
  <path class="icon-path"
        d="M4 12 L10 18 L20 6"/>
</svg>

<style>
.icon { width: 48px; height: 48px; }
.icon-path {
  fill: none;
  stroke: currentColor;
  stroke-width: 2;
}
```

```
</style>
```

7. CSS Variables and `currentColor`

```
:root {  
  --icon-color: #2563eb;  
}  
  
.icon {  
  color: var(--icon-color);  
}  
  
.icon path {  
  stroke: currentColor;  
}
```

`currentColor` lets SVG follow the surrounding text/component color and is excellent for themeable icons.

8. Styling SVG from a Parent

```
.button { color: royalblue; }  
.button:hover { color: tomato; }  
.button svg { width: 24px; }
```

If the SVG uses `currentColor`, the icon automatically follows the button color.

9. JavaScript Selecting SVG

```
const svg = document.querySelector("#diagram");  
const circle = svg.querySelector("circle");  
  
circle.setAttribute("fill", "tomato");  
circle.setAttribute("cx", "120");
```

Inline SVG elements are normal DOM elements and can be selected and modified.

10. `setAttribute` vs CSS

```
circle.setAttribute("fill", "red");  
circle.style.opacity = "0.5";  
circle.classList.add("active");
```

For UI state, classes are often cleaner than repeatedly changing inline styles.

11. Creating SVG with JavaScript

```
const svgNS = "http://www.w3.org/2000/svg";  
  
const circle =  
  document.createElementNS(svgNS, "circle");  
  
circle.setAttribute("cx", "50");  
circle.setAttribute("cy", "50");  
circle.setAttribute("r", "20");  
  
svg.appendChild(circle);
```

Programmatic SVG creation uses the SVG namespace.

12. Event Handling

```
document.querySelectorAll(".point")  
  .forEach(point => {  
    point.addEventListener("click", () => {  
      point.classList.toggle("selected");  
    });  
  });
```

13. Responsive SVG

```
<svg viewBox="0 0 500 300"
      width="100%"
      height="auto">
  ...
</svg>
```

The `viewBox` defines the internal coordinate system while CSS controls the displayed size.

14. `preserveAspectRatio`

```
<svg viewBox="0 0 100 100"
      preserveAspectRatio="xMidYMid meet">
  ...
</svg>
```

meet keeps the complete artwork visible. **slice** fills the viewport and can crop the artwork.

15. Responsive Container

```
.graphic svg {
  display: block;
  width: 100%;
  height: auto;
}
```

16. Accessibility Basics

- Decorative SVG should be hidden from assistive technology when appropriate.
- Meaningful SVG should have an accessible name.
- Interactive SVG needs keyboard access and meaningful state.
- Do not communicate important information through color alone.

17. Decorative SVG

```
<svg aria-hidden="true"
      focusable="false"
      viewBox="0 0 24 24">
  ...
</svg>
```

18. Meaningful SVG

```
<svg role="img"
      aria-labelledby="title desc"
      viewBox="0 0 300 150">
  <title id="title">Sales chart</title>
  <desc id="desc">
    Sales increased from January through March.
  </desc>
  ...
</svg>
```

19. Interactive SVG

```
<svg role="button"
      tabindex="0"
      aria-label="Open menu"
      viewBox="0 0 24 24">
  ...
</svg>
```

A production component should also implement keyboard activation and expose state appropriately.

20. SVG Sprites

```
<svg xmlns="http://www.w3.org/2000/svg">
  <symbol id="home" viewBox="0 0 24 24">
    <path d="M3 11 L12 3 L21 11 V21 H3 Z"/>
  </symbol>

  <symbol id="search" viewBox="0 0 24 24">
    <circle cx="11" cy="11" r="7"/>
    <path d="M16 16 L22 22"/>
  </symbol>
</svg>
```

A sprite stores reusable symbols in one SVG resource.

21. Using a Sprite Symbol

```
<svg class="icon"
  viewBox="0 0 24 24"
  aria-hidden="true">
  <use href="#home"/>
</svg>
```

22. React SVG Component

```
function CheckIcon({
  size = 24,
  strokeWidth = 2,
  className = ""
}) {
  return (
    <svg
      width={size}
      height={size}
      viewBox="0 0 24 24"
      fill="none"
      className={className}
      aria-hidden="true"
    >
      <path
        d="M4 12 L10 18 L20 6"
        stroke="currentColor"
        strokeWidth={strokeWidth}
        strokeLinecap="round"
        strokeLinejoin="round"
      />
    </svg>
  );
}
```

SVG becomes a reusable React component with props such as size, strokeWidth and className.

23. React Props for SVG

```
<CheckIcon
  size={32}
  strokeWidth={3}
  className="text-green-600"
/>
```

24. React Events

```
function ButtonIcon() {
  const handleClick = () => {
    console.log("clicked");
  };

  return (
    <svg
      viewBox="0 0 24 24"
      onClick={handleClick}
    >
```

```

        role="button"
        tabIndex={0}
      >
        <path d="M5 12 H19"/>
      </svg>
    );
  }
}

```

25. React State + SVG

```

function Toggle() {
  const [active, setActive] = useState(false);

  return (
    <svg
      viewBox="0 0 100 60"
      onClick={() => setActive(!active)}
    >
      <circle
        cx={active ? 70 : 30}
        cy="30"
        r="20"
      />
    </svg>
  );
}

```

26. Dynamic SVG from Data

```

function BarChart({ values }) {
  return (
    <svg viewBox="0 0 400 200">
      {values.map((value, index) => (
        <rect
          key={index}
          x={20 + index * 50}
          y={180 - value}
          width="30"
          height={value}
        />
      ))}
    </svg>
  );
}

```

This pattern is especially useful for dashboards and data visualizations.

27. SVG + Tailwind CSS

```

<svg
  className="w-8 h-8 text-blue-600"
  viewBox="0 0 24 24"
  fill="none">
  <path
    d="M4 12 L10 18 L20 6"
    stroke="currentColor"
    strokeWidth="2"
    strokeLinecap="round"
    strokeLinejoin="round"
  />
</svg>

```

28. Important JSX Attribute Differences

HTML SVG	JSX
class	className
stroke-width	strokeWidth
stroke-linecap	strokeLinecap
stroke-linejoin	strokeLinejoin
fill-rule	fillRule
clip-path	clipPath
stop-color	stopColor

29. External SVG Files in React

```
import logo from "../assets/logo.svg";

function Header() {
  return (
    <img src={logo} alt="CraveCart"/>
  );
}
```

Use this for static SVG images. Use an inline/component approach when you need to control the SVG's internal geometry.

30. SVG Components with Vite

```
// Example syntax when a suitable SVG-to-component
// plugin is configured:
import Logo from "../Logo.svg?react";

<Logo className="w-32"/>
```

The exact syntax depends on the Vite configuration and plugins in your project.

31. Dynamic Attributes in React

```
function Circle({ x, y, radius, color }) {
  return (
    <circle
      cx={x}
      cy={y}
      r={radius}
      fill={color}
    />
  );
}
```

32. Conditional SVG Rendering

```
{loading ? (
  <SpinnerIcon />
) : (
  <CheckIcon />
)}
```

SVG components work naturally for loading, success, error and empty states.

33. SVG for a Food App

- Restaurant and food-category icons.
- Cart, search, filter and profile icons.
- Order-status illustrations.
- Delivery tracking graphics.
- Ratings and star systems.
- Empty-cart illustrations.
- Admin dashboard charts.
- Interactive diagrams.

34. SVG Chart Example

```

<svg viewBox="0 0 400 220">
  <line x1="40" y1="180"
        x2="370" y2="180"
        stroke="gray"/>

  <polyline
    points="50,150 120,110 190,130 260,70 340,90"
    fill="none"
    stroke="currentColor"
    stroke-width="4"/>

  <circle cx="50" cy="150" r="5"/>
  <circle cx="120" cy="110" r="5"/>
  <circle cx="190" cy="130" r="5"/>
  <circle cx="260" cy="70" r="5"/>
  <circle cx="340" cy="90" r="5"/>
</svg>

```

35. SVG Security

- Treat SVG from untrusted sources carefully.
- Do not blindly inject arbitrary SVG markup.
- Sanitize SVG when accepting uploaded or externally supplied files.
- Prefer trusted assets for application UI.

SVG can contain more than simple shapes, so untrusted SVG should be handled as potentially active content.

36. SVG Optimization

- Remove unnecessary editor metadata.
- Reduce redundant path points when safe.
- Reuse repeated artwork with symbols/use.
- Remove unused definitions.
- Optimize files before shipping when appropriate.
- Do not remove important accessibility information during optimization.

37. Debugging SVG

- Inspect the SVG in browser DevTools.
- Check viewBox and viewport dimensions.
- Temporarily add outlines/backgrounds.
- Inspect computed CSS.
- Check clipping and overflow.
- Check fill, stroke and opacity.
- Check IDs used by url(#...) references.
- Check the console for JavaScript errors.

38. Common React SVG Errors

- Using class instead of className.

- Using HTML-style hyphenated attributes instead of JSX names.
- Passing strings where numeric calculations are expected.
- Generating duplicate IDs for gradients or clip paths.
- Changing SVG state directly when React state should control it.
- Rendering expensive animations through excessive React updates.

39. Mini Project — Reusable Icon Button

```
function IconButton({ label, children, onClick }) {
  return (
    <button
      type="button"
      aria-label={label}
      onClick={onClick}
      className="p-2"
    >
      {children}
    </button>
  );
}

function SearchIcon() {
  return (
    <svg
      viewBox="0 0 24 24"
      className="w-6 h-6"
      fill="none"
    >
      <circle
        cx="11" cy="11" r="7"
        stroke="currentColor"
        strokeWidth="2"
      />
      <path
        d="M16 16 L22 22"
        stroke="currentColor"
        strokeWidth="2"
      />
    </svg>
  );
}
```

40. Mini Project — Dynamic Star Rating

```
function Rating({ value }) {
  return (
    <svg viewBox="0 0 500 100" width="250">
      {[0, 1, 2, 3, 4].map(i => (
        <path
          key={i}
          d="M50 5 L61 38 L95 38
            L68 58 L78 92
            L50 72 L22 92
            L32 58 L5 38 L39 38 Z"
          transform={`translate(${i * 100} 0)`}
          fill={i < value ? "gold" : "lightgray"}
        />
      ))}
    </svg>
  );
}
```

For production, also provide a textual rating for users who cannot perceive the visual stars.

41. Mini Project — CraveCart Order Status

```
function OrderStatus({ status }) {
  const steps = ["Placed", "Confirmed", "Delivered"];
  // ...
}
```

```

return (
  <svg viewBox="0 0 600 120">
    {steps.map((step, i) => (
      <g key={step}>
        <circle
          cx={100 + i * 200}
          cy="60"
          r="22"
          fill={i <= status ? "seagreen" : "lightgray"}
        />
        <text
          x={100 + i * 200}
          y="100"
          textAnchor="middle"
        >
          {step}
        </text>
      </g>
    ))}
  </svg>
);
}

```

42. Mini Project — Responsive Dashboard Graphic

```

function SalesChart({ points }) {
  const pointString = points
    .map(p => `${p.x},${p.y}`)
    .join(" ");

  return (
    <svg viewBox="0 0 500 250" className="w-full">
      <polyline
        points={pointString}
        fill="none"
        stroke="currentColor"
        strokeWidth="4"
      />
    </svg>
  );
}

```

43. Production Checklist

- Does the SVG have an appropriate viewBox?
- Is it inline or external for a deliberate reason?
- Is the graphic decorative or meaningful?
- Is accessibility handled correctly?
- Are IDs unique?
- Are repeated elements reused?
- Are animations necessary and accessible?
- Is reduced-motion behavior considered?
- Does it work at different sizes?
- Has unnecessary SVG data been optimized?

44. Learning Roadmap — You Are Here

- Parts 1–2: SVG fundamentals and shapes.
- Parts 3–5: text, styling, gradients and advanced drawing.

- Part 6: clipping, masks and filters.
- Part 7: reuse, symbols and markers.
- Part 8: paths in depth.
- Part 9: animation and interaction.
- **Part 10: real web integration with HTML, CSS, JavaScript and React.**

45. Exercises

- Convert an existing HTML icon into inline SVG.
- Build a reusable React icon component with size and className props.
- Create a responsive SVG banner.
- Build a five-star rating component.
- Create a loading spinner with reduced-motion support.
- Build an interactive SVG order-status tracker.
- Create a small admin-dashboard line chart.
- Create an SVG sprite containing five UI icons.
- Make a themeable icon set using `currentColor`.
- Take an SVG from a design tool and clean it manually.

46. Quick Reference

Static logo	-> <code></code> with SVG file
Interactive icon	-> Inline SVG / React component
Reusable icon system	-> Symbols + <code><use></code> or React components
Hover effect	-> CSS transition
Complex UI animation	-> CSS / JavaScript
Path drawing	-> <code>stroke-dasharray</code> + <code>dashoffset</code>
Dynamic chart	-> React + SVG
Responsive graphic	-> <code>viewBox</code> + responsive CSS
Decorative SVG	-> <code>aria-hidden</code>
Meaningful SVG	-> <code>title/description</code> + accessible name

47. What's Next?

Part 11 will move into **Advanced SVG & Professional Techniques**: coordinate systems in depth, nested `viewBoxes`, transforms, path geometry, advanced gradients, filter composition, SVG optimization, reusable architecture, accessibility patterns and larger real-world projects.